

tf.compat.v1.distributions.Distribution

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distributions/Distribution

A generic probability distribution base class.

Function Signatures

```
tf.compat.v1.distributions.Distribution( dtype,  
    reparameterization_type, validate_args, allow_nan_stats,  
    parameters=None, graph_parents=None, name=None )  
log_prob(value, name="log_prob")  
_log_prob(value)
```

Code Examples

```
tf.compat.v1.distributions.Distribution(  
    dtype,  
    reparameterization_type,  
    validate_args,  
    allow_nan_stats,  
    parameters=None,  
    graph_parents=None,  
    name=None  
)
```

```
tf.compat.v1.distributions.Distribution(  
dtype,  
reparameterization_type,  
validate_args,  
allow_nan_stats,  
parameters=None,  
graph_parents=None,  
name=None  
)
```

```
@util.AppendDocstring("Some other details.")  
def _log_prob(self, value):  
...
```

```
@util.AppendDocstring("Some other details.")  
def _log_prob(self, value):  
...
```

```

minval = 3.0
maxval = [[4.0, 6.0],
[10.0, 12.0]]
# Broadcasting:
# This instance represents 4 Uniform distributions. Each has a
lower bound at
# 3.0 as the `minval` parameter was broadcasted to match `maxval`'s
shape.
u = Uniform(minval, maxval)
# `event_shape` is `TensorShape([])`.
event_shape = u.event_shape
# `event_shape_t` is a `Tensor` which will evaluate to [].
event_shape_t = u.event_shape_tensor()
# Sampling returns a sample per distribution. `samples` has shape
# [5, 2, 2], which is [n] + batch_shape + event_shape, where n=5,
# batch_shape=[2, 2], and event_shape=[].
samples = u.sample_n(5)
# The broadcasting holds across methods. Here we use `cdf` as an
example. The
# same holds for `log_cdf` and the likelihood functions.
# `cum_prob` has shape [2, 2] as the `value` argument was
broadcasted to the
# shape of the `Uniform` instance.
cum_prob_broadcast = u.cdf(4.0)
# `cum_prob`'s shape is [2, 2], one per distribution. No
broadcasting
# occurred.
cum_prob_per_dist = u.cdf([[4.0, 5.0],
[6.0, 7.0]])
# INVALID as the `value` argument is not broadcastable to the
distribution's
# shape.
cum_prob_invalid = u.cdf([4.0, 5.0, 6.0])

```

```

minval = 3.0
maxval = [[4.0, 6.0],
[10.0, 12.0]]
# Broadcasting:
# This instance represents 4 Uniform distributions. Each has a
lower bound at
# 3.0 as the `minval` parameter was broadcasted to match `maxval`'s
shape.
u = Uniform(minval, maxval)
# `event_shape` is `TensorShape([])`.
event_shape = u.event_shape
# `event_shape_t` is a `Tensor` which will evaluate to [].
event_shape_t = u.event_shape_tensor()
# Sampling returns a sample per distribution. `samples` has shape
# [5, 2, 2], which is [n] + batch_shape + event_shape, where n=5,
# batch_shape=[2, 2], and event_shape=[].
samples = u.sample_n(5)
# The broadcasting holds across methods. Here we use `cdf` as an
example. The
# same holds for `log_cdf` and the likelihood functions.
# `cum_prob` has shape [2, 2] as the `value` argument was
broadcasted to the
# shape of the `Uniform` instance.
cum_prob_broadcast = u.cdf(4.0)
# `cum_prob`'s shape is [2, 2], one per distribution. No
broadcasting
# occurred.
cum_prob_per_dist = u.cdf([[4.0, 5.0],
[6.0, 7.0]])
# INVALID as the `value` argument is not broadcastable to the
distribution's
# shape.
cum_prob_invalid = u.cdf([4.0, 5.0, 6.0])

```

```
a = tf.exp(tf.matmul(logits, weights_a))
b = tf.exp(tf.matmul(logits, weights_b))
# Will raise exception if ANY batch member has a < 1 or b < 1.
dist = distributions.beta(a, b, allow_nan_stats=False)
mode = dist.mode().eval()
# Will return NaN for batch members with either a < 1 or b < 1.
dist = distributions.beta(a, b, allow_nan_stats=True) # Default
behavior
mode = dist.mode().eval()
```

```
a = tf.exp(tf.matmul(logits, weights_a))
b = tf.exp(tf.matmul(logits, weights_b))
# Will raise exception if ANY batch member has a < 1 or b < 1.
dist = distributions.beta(a, b, allow_nan_stats=False)
mode = dist.mode().eval()
# Will return NaN for batch members with either a < 1 or b < 1.
dist = distributions.beta(a, b, allow_nan_stats=True) # Default
behavior
mode = dist.mode().eval()
```

Documentation

A generic probability distribution base class.

Distribution is a base class for constructing and organizing properties (e.g., mean, variance) of random variables (e.g, Bernoulli, Gaussian).

Subclassing

Subclasses are expected to implement a leading-underscore version of the same-named function. The argument signature should be identical except for the omission of `name="..."`. For example, to enable `log_prob(value, name="log_prob")` a subclass should implement `_log_prob(value)`.

Subclasses can append to public-level docstrings by providing docstrings for their method specializations. For example:

would add the string "Some other details." to the `log_prob` function docstring. This is implemented as a simple decorator to avoid python linter complaining about missing Args/Returns/Raises sections in the partial docstrings.

Broadcasting, batching, and shapes

All distributions support batches of independent distributions of that type. The batch shape is determined by broadcasting together the parameters.

The shape of arguments to `__init__`, `cdf`, `log_cdf`, `prob`, and `log_prob` reflect this broadcasting, as does the return value of `sample` and `sample_n`.

$\text{sample_n_shape} = [n] + \text{batch_shape} + \text{event_shape}$, where `sample_n_shape` is the shape of the Tensor returned from `sample_n`, `n` is the number of samples, `batch_shape` defines how many independent distributions there are, and `event_shape` defines the shape of samples from each of those independent distributions. Samples are independent along the `batch_shape` dimensions, but not necessarily so along the `event_shape` dimensions (depending on the particulars of the underlying distribution).

Using the Uniform distribution as an example:

Shapes

There are three important concepts associated with TensorFlow Distributions shapes:

- Event shape describes the shape of a single draw from the distribution;

it may be dependent across dimensions. For scalar distributions, the event shape is []. For a 5-dimensional MultivariateNormal, the event shape is [5].

- Batch shape describes independent, not identically distributed draws, aka a

"collection" or "bunch" of distributions.

- Sample shape describes independent, identically distributed draws of batches

from the distribution family.

The event shape and the batch shape are properties of a Distribution object, whereas the sample shape is associated with a specific call to sample or log_prob.

For detailed usage examples of TensorFlow Distributions shapes, see [this tutorial](#)

Parameter values leading to undefined statistics or distributions.

Some distributions do not have well-defined statistics for all initialization parameter values. For example, the beta distribution is parameterized by positive real numbers concentration1 and concentration0, and does not have well-defined mode if $\text{concentration1} < 1$ or $\text{concentration0} < 1$.

The user is given the option of raising an exception or returning NaN.

In all cases, an exception is raised if invalid parameters are passed, e.g.

Raises

Attributes

Stats return +/- infinity when it makes sense. E.g., the variance of a Cauchy distribution is infinity. However, sometimes the statistic is undefined, e.g., if a distribution's pdf does not achieve a maximum within the support of the distribution, the mode is undefined. If the mean is undefined, then by definition the variance is undefined. E.g. the mean for Student's T for $df = 1$ is undefined (no clear way to say it is either + or - infinity), so the variance = $E[(X - \text{mean})^2]$ is also undefined.

May be partially defined or unknown.

The batch dimensions are indexes into independent, non-identical parameterizations of this distribution.

May be partially defined or unknown.

Currently this is one of the static instances
`distributions.FULLY_REPARAMETERIZED`
or `distributions.NOT_REPARAMETERIZED`.

Methods

`## batch_shape_tensor`

[View source](#)

Shape of a single sample from a single event index as a 1-D Tensor.

The batch dimensions are indexes into independent, non-identical parameterizations of this distribution.

[View source](#)

Cumulative distribution function.

Given random variable X , the cumulative distribution function cdf is:

[View source](#)

Creates a deep copy of the distribution.

covariance

[View source](#)

Covariance.

Covariance is (possibly) defined only for non-scalar-event distributions.

For example, for a length- k , vector-valued distribution, it is calculated as,

where Cov is a (batch of) $k \times k$ matrix, $0 \leq (i, j) < k$, and E denotes expectation.

Alternatively, for non-vector, multivariate distributions (e.g., matrix-valued, Wishart), Covariance shall return a (batch of) matrices under some vectorization of the events, i.e.,

where Cov is a (batch of) $k' \times k'$ matrices, $0 \leq (i, j) < k' = \text{reduce_prod}(\text{event_shape})$, and Vec is some function mapping indices of this distribution's event dimensions to indices of a

length-k' vector.

cross_entropy

[View source](#)

Computes the (Shannon) cross entropy.

Denote this distribution (self) by P and the other distribution by Q . Assuming P, Q are absolutely continuous with respect to one another and permit densities $p(x) dx$ and $q(x) dx$, (Shanon) cross entropy is defined as:

where F denotes the support of the random variable $X \sim P$.

entropy

[View source](#)

Shannon entropy in nats.

event_shape_tensor

[View source](#)

Shape of a single sample from a single batch as a 1-D int32 Tensor.

is_scalar_batch

[View source](#)

Indicates that `batch_shape == []`.

is_scalar_event

[View source](#)

Indicates that `event_shape == []`.

kl_divergence

[View source](#)

Computes the Kullback--Leibler divergence.

Denote this distribution (self) by p and the other distribution by q . Assuming p, q are absolutely continuous with respect to reference measure r , the KL divergence is defined as:

where F denotes the support of the random variable $X \sim p$, $H[., .]$ denotes (Shanon) cross entropy, and $H[.]$ denotes (Shanon) entropy.

log_cdf

[View source](#)

Log cumulative distribution functi